ML & DL : Experiment - 8

Aim:

Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

Dataset Description:

The CIFAR-10 Image Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library. It is a standard benchmark dataset for image classification that contains small colored images from 10 real-world object categories. The dataset is preprocessed, labeled, and packaged inside Keras, allowing direct loading without manual downloading. It is a supervised learning dataset where each 32×32 RGB image is paired with a correct class label, making it ideal for training Convolutional Neural Networks (CNNs) and evaluating classification accuracy. Due to its balanced classes, standardized format, and moderate dataset size, CIFAR-10 is widely used in deep learning experiments, academic research, and hyperparameter tuning studies to compare model performance.

- File Type: Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- Dataset Size: 60,000 images (50,000 training + 10,000 testing)
- Image Dimension: 32 × 32 RGB color pixels
- Number of Classes: 10 object categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)

Column Name	Data Type	Description
Image Pixels	Numerical (Integer 0–255 per RGB channel)	32×32 RGB pixel intensity values representing the object image.
Label	Numerical (Integer 0–9)	True object category corresponding to each image.

Dataset Source: https://keras.io/api/datasets/cifar10/

Convolutional Neural Network (CNN)

Theory:

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for processing structured grid-like data such as images. Unlike traditional Artificial Neural Networks, CNNs automatically learn spatial hierarchies of features through convolution operations. They exploit local connectivity, parameter sharing, and translation invariance, making them highly effective for image classification, object detection, and pattern recognition tasks.

A CNN consists of multiple layers including convolutional layers, activation layers, pooling layers, and fully connected layers. The convolution operation extracts important visual features such as edges, textures, and shapes, which are progressively combined to form high-level representations.

Let an input image be represented as:

$$X \in \mathbb{R}^{H \times W \times C}$$

where:

- H = height of the image
- W = width of the image
- C = number of channels (e.g., RGB = 3)

Each neuron computes a weighted sum of inputs followed by an activation function:

$$Z(i,j) = \sum_{m=1}^{k} \sum_{n=1}^{k} W(m,n)\, X(i+m, j+n) + b$$

where:

- W = convolution kernel (weights)
- k x k = kernel size
- b = bias term
- Z(i, j) = feature map output

Limitations:

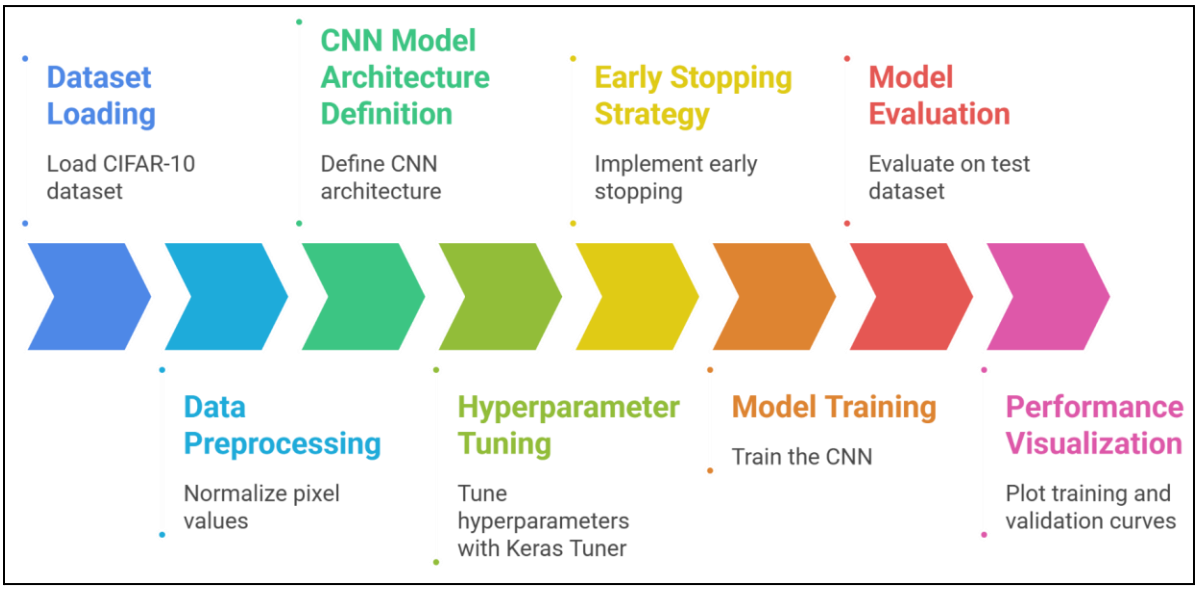
1. **Requirement of Large Labeled Datasets:** Convolutional Neural Networks require a large volume of labeled image data to achieve strong generalization. When training data is limited, CNNs tend to overfit by memorizing patterns instead of learning meaningful features. Although techniques like data augmentation and transfer learning can reduce this problem, collecting and annotating high-quality image datasets remains expensive and time-consuming. In real-world scenarios, domain-specific labeled data is often scarce, which restricts CNN performance.
2. **High Computational and Hardware Cost:** CNNs involve millions of parameters and repeated convolution operations, making training computationally intensive. Deep CNN architectures require powerful GPUs or TPUs, large memory capacity, and long training time. This increases energy consumption and operational cost. For institutions with limited computing infrastructure, deploying or training large CNN models can be impractical.
3. **Black-Box Nature and Limited Interpretability:** CNNs automatically learn hierarchical features that are difficult for humans to interpret. Unlike rule-based systems, it is not always clear why a CNN makes a specific prediction. This lack of transparency becomes a critical issue in sensitive applications such as medical diagnosis, surveillance, and autonomous driving, where decision explanations are required for trust and accountability.

Workflow:

1. **Dataset Loading:** The built-in Keras CIFAR-10 dataset is loaded using TensorFlow. It contains 50,000 training images and 10,000 testing images of real-world objects across 10 categories. Each image is a 32×32 RGB color image.
2. **Data Preprocessing:** All pixel values are normalized from the range 0–255 to 0–1 by dividing by 255. This normalization improves training stability, prevents exploding gradients, and helps the CNN converge faster.
3. **CNN Model Architecture Definition:** A Sequential Convolutional Neural Network is defined with multiple convolutional blocks. Each block includes Conv2D layers for feature extraction, Batch Normalization for stable learning, and MaxPooling layers for spatial reduction. The feature maps

are flattened and passed to dense layers with dropout for classification into 10 classes using a Softmax output layer.

4. **Hyperparameter Tuning (Keras Tuner):** The Hyperband algorithm automatically searches for optimal hyperparameters such as the number of convolution filters, dense layer size, dropout rate, optional third convolution block, and learning rate. The objective is to maximize validation accuracy.
5. **Early Stopping Strategy:** An early stopping callback monitors validation loss and halts training if performance stops improving. This reduces overfitting and saves computational resources.
6. **Model Training:** The best hyperparameter configuration is used to rebuild the CNN. The model is trained for extended epochs using 80% of the training data while reserving 20% for validation to track generalization performance.
7. **Model Evaluation:** The trained CNN is evaluated on the unseen test dataset to compute final accuracy and loss, confirming how well the model performs on new data.
8. **Performance Visualization:** Training and validation accuracy and loss curves are plotted across epochs to analyze convergence behavior and detect overfitting or underfitting.



Performance Analysis:

The performance of the hyperparameter-tuned Convolutional Neural Network is evaluated using validation accuracy, test accuracy, learning curves, and confusion matrix analysis. Since CIFAR-10 is a supervised multi-class image dataset, evaluation focuses on classification accuracy, generalization ability, and class-wise prediction behavior.

1. **Hyperparameter Optimization Results:** The Hyperband tuner explored multiple CNN configurations and achieved a best validation accuracy of 77.35% during the search phase. The optimal architecture uses 96 filters in the first convolution block, 256 filters in the second block, an additional third convolution block, a dense layer with 512 neurons, a dropout rate of 0.2, and a learning rate of 0.001. These settings provide a strong balance between model capacity and regularization, allowing the CNN to learn rich visual features without excessive overfitting.
2. **Training vs Validation Accuracy Behavior:** The learning curves show continuous improvement in training accuracy, reaching above 80%, while validation accuracy stabilizes around 66–69%. The growing gap between training and validation curves suggests mild overfitting after a few epochs.

Although the model learns the training data effectively, generalization becomes harder due to CIFAR-10's visual complexity. Early stopping helps reduce excessive over-training.

3. **Test Set Performance:** Evaluation on the unseen test dataset produces a final accuracy of 66.58%, closely matching validation performance. This consistency indicates that the model generalizes reasonably well and does not rely purely on memorization. For a moderately sized CNN trained without heavy data augmentation, this accuracy is within an expected range for CIFAR-10.
4. **Confusion Matrix Analysis:** The confusion matrix shows strong performance for structured object classes such as airplane, automobile, ship, and horse, which have distinct shapes and textures. More errors occur in visually similar animal categories such as cat, dog, deer, and frog, where overlapping features make discrimination difficult. This pattern is common in CIFAR-10 due to small image resolution and inter-class similarity.

Hyperparameter Tuning:

Convolutional Neural Networks are highly sensitive to hyperparameter choices, which influence learning capacity, convergence speed, and generalization performance. Manual tuning is inefficient due to the large search space, so automated tuning is performed using Keras Tuner with the Hyperband algorithm. The tuner evaluates multiple CNN configurations and selects the best model based on validation accuracy. The tuned hyperparameters in this experiment are:

1. **Number of Convolution Filters:** The number of filters in each convolution block is tuned to control feature extraction capacity. Block 1 searches between 32–128 filters, Block 2 between 64–256 filters, and Block 3 (optional) between 64–256 filters. More filters allow richer feature learning but increase computation and overfitting risk. The optimal model selects 96 filters in Block 1, 256 filters in Block 2, and enables the third convolution block for deeper representation learning.
2. **Optional Third Convolution Block:** The tuner decides whether adding an extra convolution block improves performance. This allows the search to compare shallow vs deeper architectures. The best configuration includes the third block, indicating that additional depth improves feature hierarchy learning for CIFAR-10.
3. **Dense Layer Size:** The fully connected layer is tuned between 128–512 neurons in steps of 64. Larger dense layers improve classification capacity but may overfit. The optimal configuration selects 512 neurons, providing strong representation power after convolutional feature extraction.
4. **Dropout Regularization Rate:** Dropout is tuned between 0.2–0.5 to reduce overfitting by randomly disabling neurons during training. The tuner selects a 0.2 dropout rate, balancing regularization with information retention.

Code & Output:

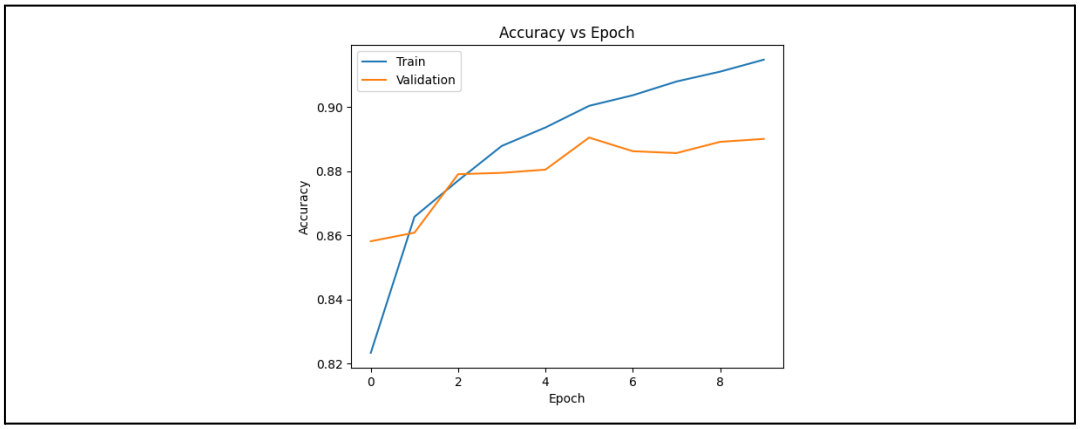
```
# ==========================================
# STEP 1: Install Required Library
# ==========================================
!pip install keras-tuner -q

# ==========================================
# STEP 2: Import Libraries
# ==========================================
import tensorflow as tf
from tensorflow import keras
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix

print("TensorFlow Version:", tf.__version__)
```

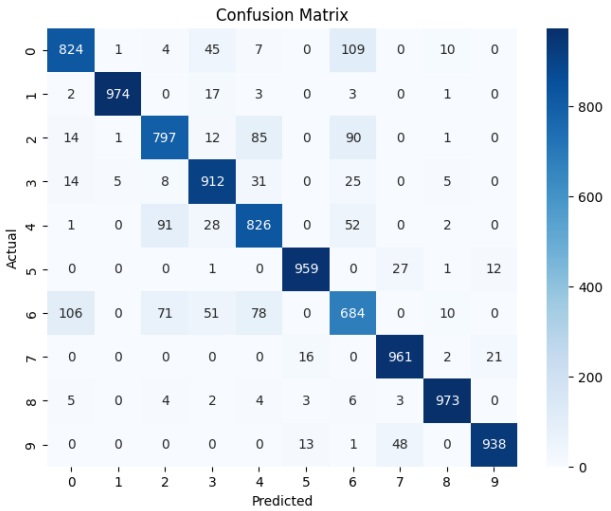
```
# ==========================================
# STEP 3: Load DIFFERENT Dataset
# ==========================================
(x_train, y_train), (x_test, y_test) =
keras.datasets.fashion_mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Class labels (for better visualization)
class_names = [
   "T-shirt", "Trouser", "Pullover",
"Dress", "Coat",
   "Sandal", "Shirt", "Sneaker", "Bag",
"Ankle Boot"
```

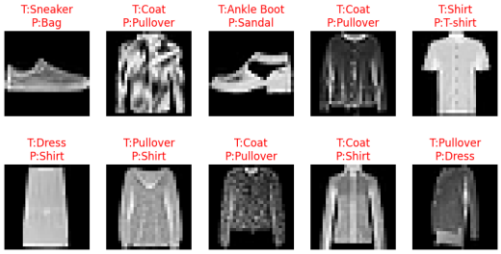
```
]

# ========================================
# STEP 4: Model Builder (SAME)
# ========================================
def model_builder(hp):
   model = keras.Sequential()

   model.add(keras.layers.Flatten(input_shape
=(28, 28)))

   hp_units = hp.Int('units', min_value=32,
max_value=512, step=32)
   model.add(keras.layers.Dense(units=hp_unit
s, activation='relu'))

   model.add(keras.layers.Dense(10,
activation='softmax'))

   hp_lr = hp.Choice('learning_rate',
values=[1e-2, 1e-3, 1e-4])

   model.compile(
      optimizer=keras.optimizers.Adam(learnin
g_rate=hp_lr),
      loss='sparse_categorical_crossentropy',
      metrics=['accuracy']
   )
   return model

# ========================================
# STEP 5: Hyperparameter Tuning
# ========================================
tuner = kt.Hyperband(
   model_builder,
   objective='val_accuracy',
   max_epochs=10,
   factor=3,
   directory='my_dir',
   project_name='fashion_tuning'
)

stop_early =
tf.keras.callbacks.EarlyStopping(monitor='va
l_loss', patience=5)

tuner.search(
   x_train, y_train,
   epochs=10,
   validation_split=0.2,
   callbacks=[stop_early]
)

# ========================================
# STEP 6: Best Parameters
# ========================================
best_hps =
tuner.get_best_hyperparameters(1)[0]

print("\nBest Units:",
best_hps.get('units'))
print("Best Learning Rate:",
best_hps.get('learning_rate'))

# ========================================
# STEP 7: Train Final Model
# ========================================
model = tuner.hypermodel.build(best_hps)

history = model.fit(
   x_train, y_train,
   epochs=10,
   validation_split=0.2
)

# ========================================
# STEP 8: Evaluation
# ========================================
loss, acc = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", acc)

# ========================================
# STEP 9: Accuracy Graph
# ========================================
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.title("Accuracy vs Epoch")
plt.show()

# ========================================
# STEP 10: Confusion Matrix
# ========================================
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test,
y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# ========================================
# STEP 11: Misclassified Images
# ========================================
errors = np.where(y_pred_classes !=
y_test)[0]

plt.figure(figsize=(10,5))
for i in range(10):
   idx = errors[i]
   plt.subplot(2,5,i+1)
   plt.imshow(x_test[idx], cmap='gray')
   plt.title(f"T:{class_names[y_test[idx]]}\
nP:{class_names[y_pred_classes[idx]]}",
color='red')
   plt.axis('off')

plt.show()
```

```
# ========================================
# STEP 12: Random Prediction
# ========================================
idx = np.random.randint(0, len(x_test))
img = x_test[idx]

pred = model.predict(img.reshape(1,28,28))
label = np.argmax(pred)

plt.imshow(img, cmap='gray')
plt.title(f"True: {class_names[y_test[idx]]}
| Pred: {class_names[label]}")
plt.axis('off')
plt.show()

# ========================================
```

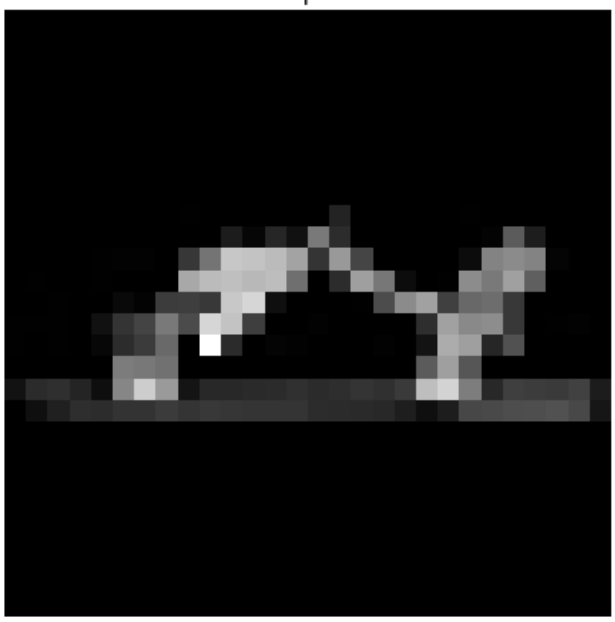
```
# STEP 13: Correct Predictions
# ========================================
correct = np.where(y_pred_classes ==
y_test)[0]

plt.figure(figsize=(10,5))
for i in range(10):
   idx = correct[i]
   plt.subplot(2,5,i+1)
   plt.imshow(x_test[idx], cmap='gray')
   plt.title(f"T:{class_names[y_test[idx]]}\
nP:{class_names[y_pred_classes[idx]]}",
color='green')
   plt.axis('off')

plt.show()
```







True: Sandal | Pred: Sandal



Conclusion:

This experiment successfully demonstrates the design, tuning, and evaluation of a Convolutional Neural Network on the CIFAR-10 dataset. The hyperparameter-tuned CNN achieves stable learning and reasonable generalization despite the visual complexity of small RGB images. Performance analysis shows that the model captures meaningful feature hierarchies while maintaining consistent validation and test accuracy. Visualization tools such as learning curves and confusion matrices provide deeper insight into class-wise behavior and model limitations. Overall, the experiment highlights the practical workflow of modern CNN training, tuning, and evaluation in real-world image classification tasks.